The commented out portion shows an example record.

[RECORD 1] shows that 5122 distinct first names are captured in the n_distinct column.

You can see some of the most common first names like Elroy, Maurice, and Tristan in the most_common_vals column. "Elroy" is one of the most common first names. The most_common_freqs column shows the frequency at which the most common values occur.

Calculate the frequency of Elroy as a first name. The frequency is the total number of occurrences of "Elroy" divided by the total number of rows in the users table. Since Elroy appears 405 times, divided by 1,000,000 total rows, it has a frequency of 0.000405.

The *cardinality* of a column describes the distinct values the column has.

For query performance analysis it's ideal to closely match replacement text with the original text to keep characteristics like cardinality and frequency similar to what's in production. Besides that, the space consumed within the column is worth keeping as close as possible.

"Elroy" is a string of five characters. "Elroy" is stored in a character varying column. A good replacement for Elroy would be five characters in length and stored in a column of the same type.

Another statistic for columns is the percentage of NULL and non-NULL values. PostgreSQL tracks this as the null_frac or null fraction.

NULL values in columns consume less space but are still maintained in indexes.

When replacing text content, consider the column cardinality (*What Does Cardinality Mean in a Database?*[3]), frequency distributions, and the proportion of NULL values as you design your text replacement functions.

You've now had a brief overview of some of the statistics PostgreSQL tracks about the data.

As you begin to identify sensitive columns like email that you want to scrub, how will you keep track of all the columns over time?

3. https://vertabelo.com/blog/cardinality-in-database/